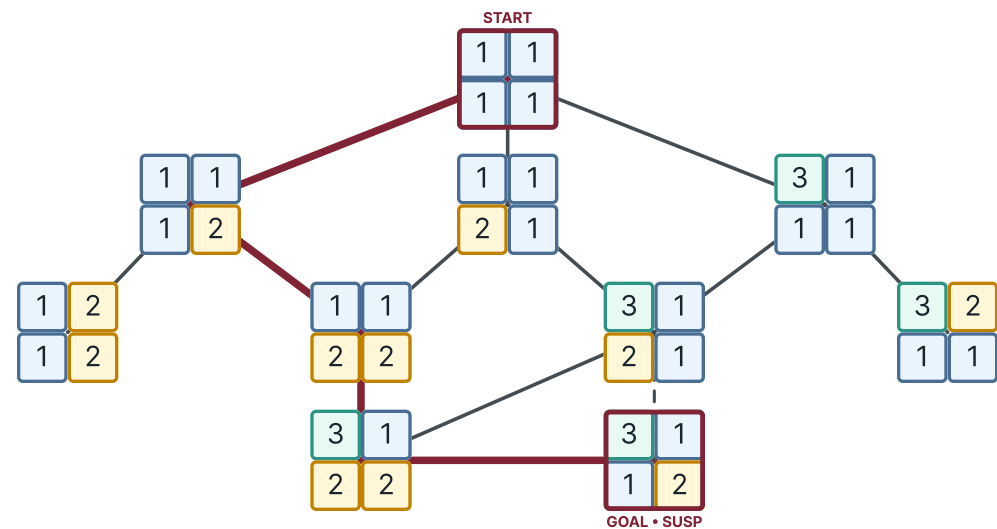


CPU-GPU Synchronization Stalls Search

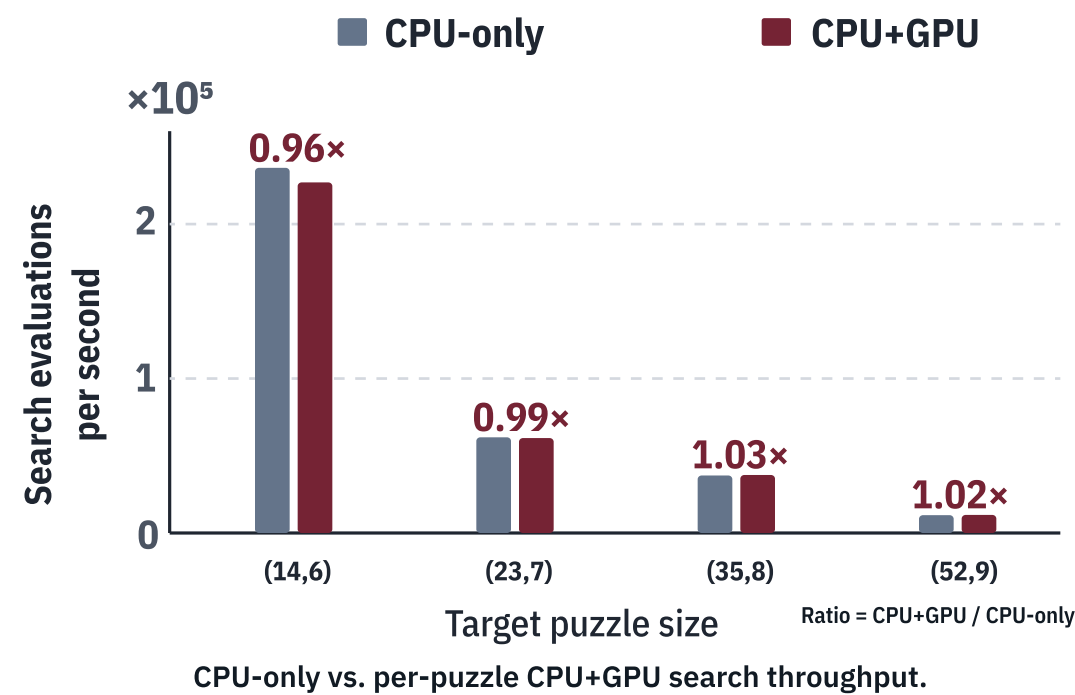
Motivation: A puzzle is a 2D array whose elements are chosen from $\{1,2,3\}$. **SUSPs** are **rare** puzzles with a particular combinatorial property. Larger SUSPs enable faster **matrix multiplication**, but finding them is difficult [1,2]. (*)

The problem: The existing search evaluates one candidate at a time, leaving the GPU mostly idle.

Main Contribution: A heterogeneous CPU–GPU search architecture that batches candidate evaluations and overlaps CPU and GPU work.



Naive GPU Offload Provides $\approx 1\times$ End-to-End Throughput

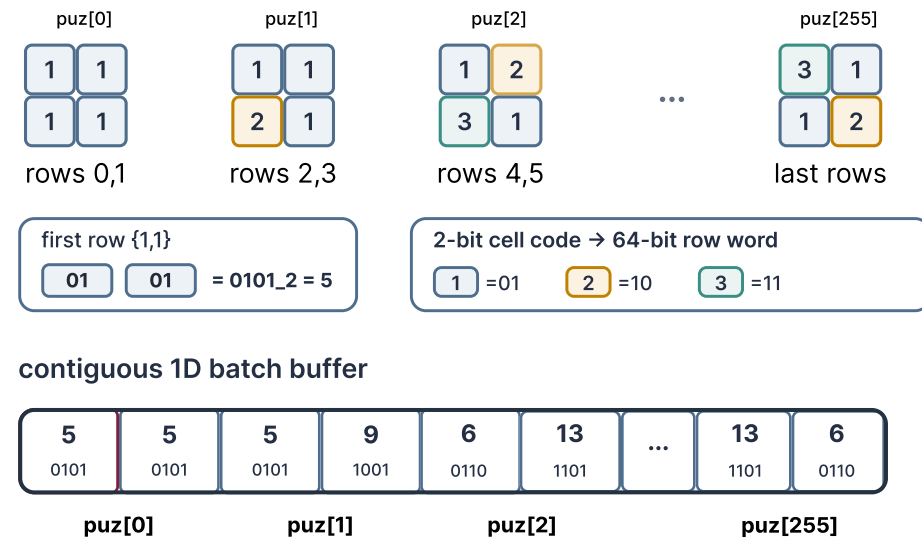


Why naive GPU offload stalls: Puzzle evaluation is **highly parallel**, but single-candidate offload repeatedly **allocates memory** and forces **CPU/GPU synchronization**.(*) OpenMP frontier workers can feed the GPU, yet **overhead** dominates, so a naive GPU path **barely changes** end-to-end throughput.

Asynchronous Queueing Decouples CPU and GPU Execution

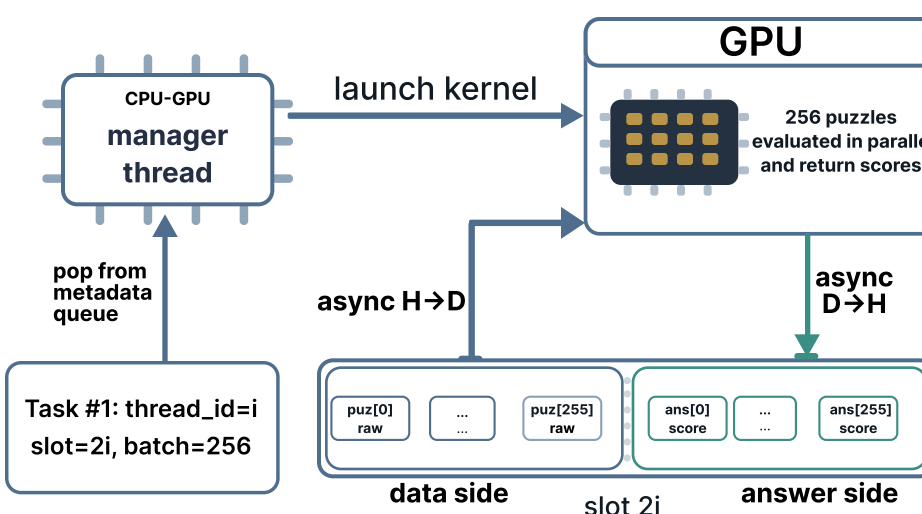
Data Serialization Maximizes Memory Bandwidth

Each cell can be $\{1, 2, 3\}$, so 2 bits encode one cell; each packed row is a 64-bit integer.



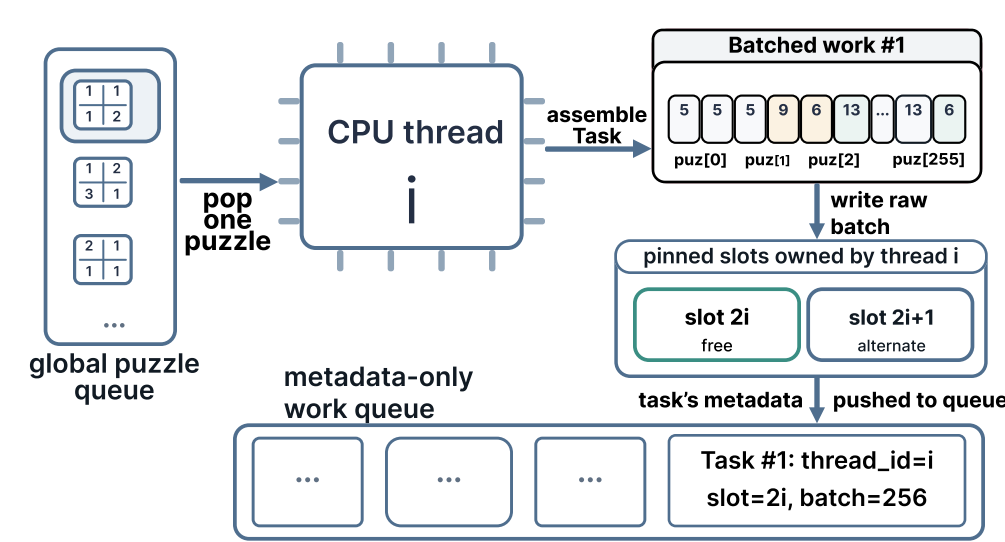
Dedicated GPU Manager Hides Transfer Latency 3

metadata → slot 2i → async H2D → kernels → async D2H answers



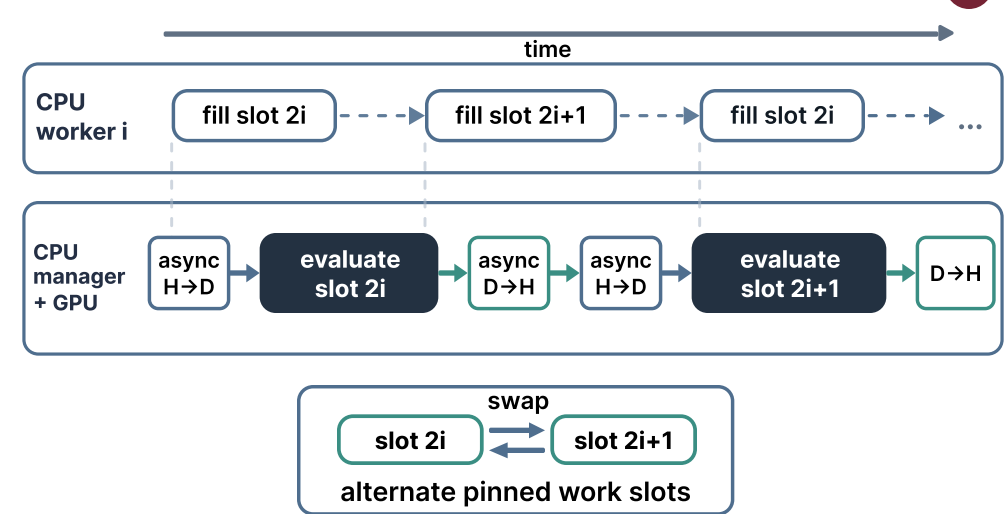
CPU Workers Build and Queue Batches

Dense puzzle → Task → slot $2i/2i+1$ → metadata-only queue

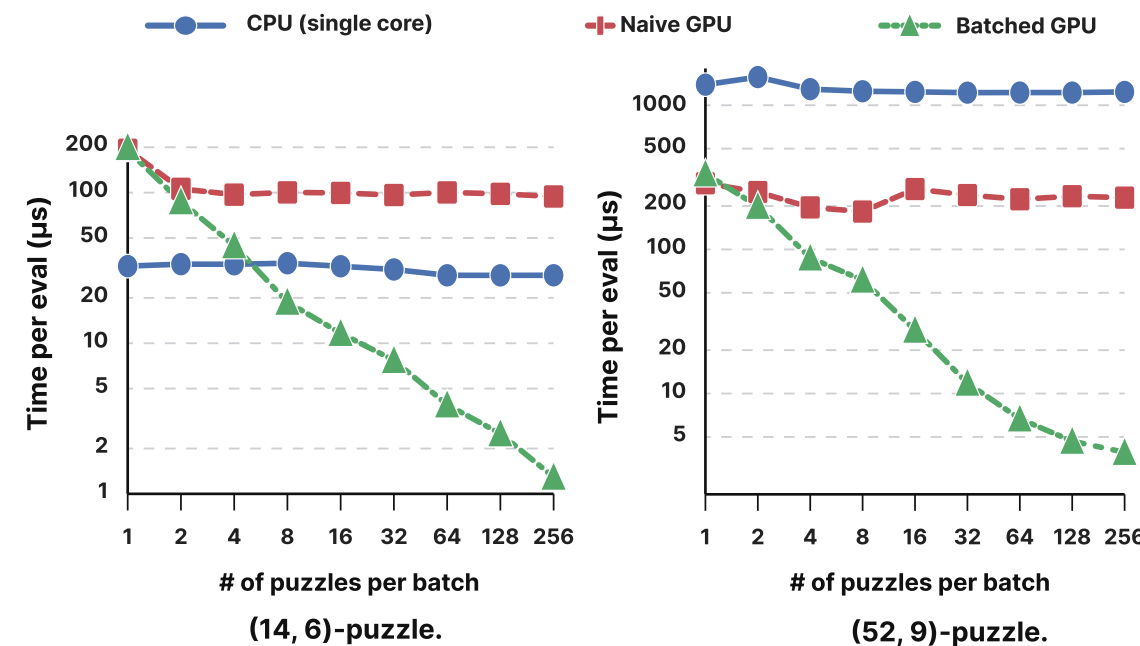


Double Buffering Overlaps CPU and GPU Execution

GPU evaluates slot $2i$ while worker i fills slot $2i+1$; then slots alternate



Average evaluation time per puzzle vs. batch size



New discovery: (79, 10) SUSP

Previous best: (78,10) [2]
New result: (79,10)

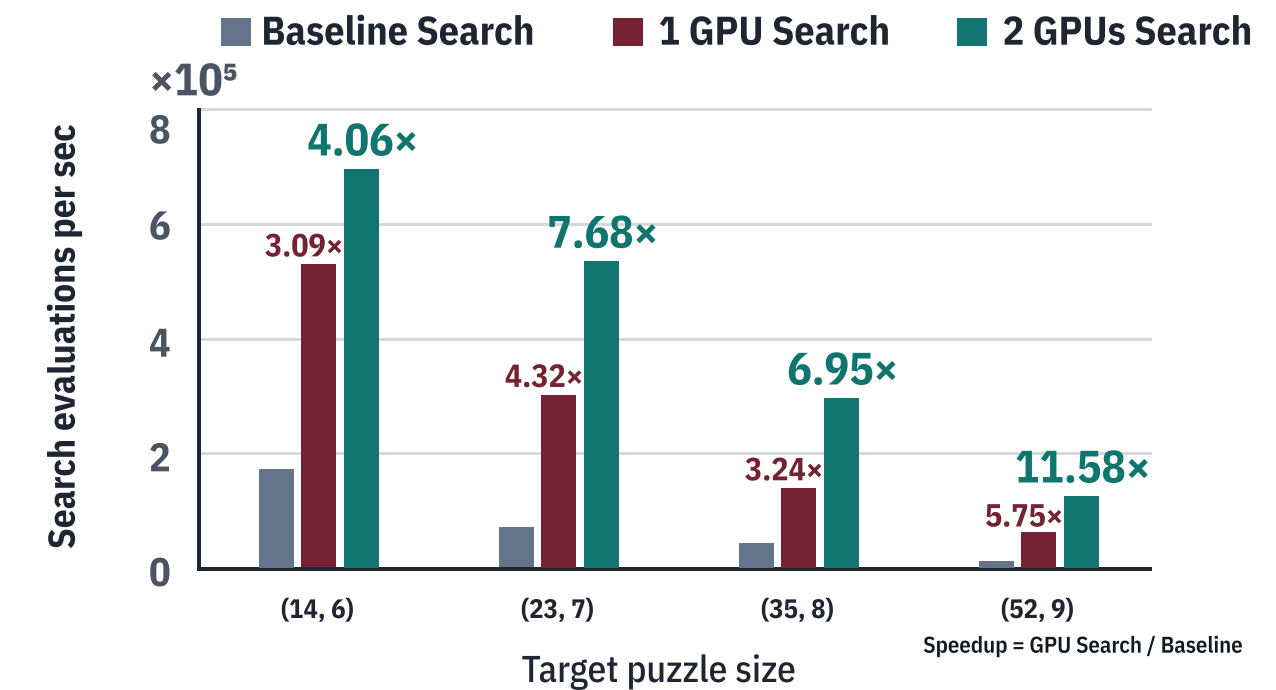
We found a **(79, 10) SUSP**, improving on the largest width-10 SUSP previously reported by Anderson and Le [2].

We started searching with Anderson and Le’s (78, 10)-SUSP as seed and used the configuration described in the Experiment Setup section.



GitHub repository
matrix-multiplication-search

Enhanced Search Raises End-to-End Throughput By 11.58x



Multi-GPU extension: This pipeline can support scale to additional GPUs to increase throughput without disrupting the existing workflow. The CPU manager thread selects GPUs in **round-robin** order and launches kernels on each device. As problem size grows and overhead becomes small relative to GPU computation time, throughput **scales nearly linearly** with the number of GPUs available in the research environment.

Related Work

CPU–GPU heterogeneous systems have been applied to combinatorial search in real-world applications, including route planning, circuit design, drug discovery, etc. FastGR combines CPU-based maze routing with GPU-based pattern routing through heterogeneous scheduling, achieving a 2.489× end-to-end speedup and a 9.324× speedup for the accelerated stage [3]. (*)

In computational drug discovery, Uni-Dock [4] docks molecules in batches to eliminate host-side bottlenecks. It achieves more than a 1000 $\times$ speedup over single-threaded AutoDock Vina when screening tens of millions of compounds, addressing a per-candidate scoring bottleneck similar to that in our work.

Conclusions & Future Work

We developed a heterogeneous search framework with batched evaluation, multiple CPU frontier-expansion workers, and a dedicated GPU manager to keep the CPU and GPU busy. Future work will scale the search across distributed multi-node CPU–GPU systems and develop resource-aware scheduling for efficient scaling.

References & Acknowledgements

[1] Yuh C et al., "Group-theoretic algorithms for matrix multiplication," FOCS 2005.

[2] Anderson & Le, "Efficiently-verifiable SUSP and matrix multiplication," COCOMO 2001.

[3] S. Liu et al., "FastRG: Graph Routing on CPU-GPU With Heterogeneous Task Graph Scheduler," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 42, no. 7, pp. 2337-2350, July 2023, doi: 10.1109/TCAD.2022.3217668. keywords: (RoutingTask analysis; Runtime; Virus; Mirrors; Metals; Graphics; processing units; Parallel; Graphs; Routing;)

[4] Co, C. G. Cai, J. Wang, Z. Bo, Z. Zhu, and H. Zhong, "Uni-Dock: GPU-accelerated docking enables ultrahigh virtual screening," Journal of Chemical Theory and Computation, vol. 19, no. 11, pp. 3336-3345, 2019.

Acknowledgments: This work was supported by Union College's Summer Research Fellowships

Experimental Environment

- **CPU:** AMD Ryzen 9 9900X 12 cores/ 24 threads
- **GPUs:** NVIDIA GeForce RTX 5070 Ti 16 GiB x2
- **RAM:** 64 GiB
- **CPU baseline:** OpenMP, 24 threads
- **GPU batch size:** 256 candidates unless noted
- **Measurement:** wall-clock timers, mean of 10 runs